Confidential mode: what a Routstr node can and cannot see

The short version. In confidential mode, your routstrd talks to the AI provider (for example Venice) through the Routstr node. The node only relays encrypted traffic and adds its own API key. It cannot read your prompts or the answers, and it cannot change them. Billing uses the provider's own usage report. You need no new apps, accounts or special hardware (no enclaves): it is a software upgrade of routstrd and the node, switched on per request.

What you trust, and for what

Party	You trust it for	You no longer trust it for
The provider (e.g. Venice)	Running the model you asked for and handling your data per its policy; it sees your prompt, as always.	
The Routstr node	Business honesty: taking your payment, relaying traffic, refunding unused money, not cutting sessions short. Reputation covers this, as it does today.	Reading your prompts and answers, changing them, swapping the model, billing more than the provider's usage report for a completed request.
Your routstrd	Everything: it holds the keys, on your machine.	

The assumption behind this design: **nodes resell reputable providers, and you should not have to trust the node much.** Cryptography replaces most of that trust; node reputation covers the rest.

The three common objections

1. "Can the node change my tool calls, system prompt or messages?" No. Your request body (messages, system prompt, tools, tool results) is encrypted with keys only your routstrd has; if the node changed one byte, the provider would reject it. Likewise your routstrd checks every piece of the answer, tool calls included, so a modified answer is detected and discarded. The node writes only at the start, before the keys switch to ones only your routstrd holds: the request header with its API key, which a zero-knowledge proof shows matches a published template. It could still write other or extra records there unseen (another account or endpoint, a header asking the provider to retain data), but none of them carries or touches your content.

2. "Can the node change the model, say to a cheaper one, or bill me for a dearer one?" No, with one exception for failed sessions (under Money). The model name and token limits sit at the end of your encrypted request, where they override anything earlier. The node holds back that last piece until your routstrd has proven it names the agreed model and limits; until then the provider has an incomplete request and does nothing. Your routstrd also checks the provider's certificate and that its hostname is on your trusted list. The bill is the provider's own usage report, which your routstrd proves to the node, at the rates the node signed beforehand; routstrd rejects a signed receipt that does not match.

3. "Can the Routstr node read my prompts and responses?" No. It only handles ciphertext; the keys never leave your machine. It still sees which provider and model you use, when, how long your request is, and the size of each streamed chunk, which can leak some of an answer (research has partially reconstructed responses from it), far less than today, when it sees everything. On a provider error it sees only the status and headers.

What it costs, and the limits

- **Speed and data:** each request first runs a two-party computation with the node: routstrd downloads about 22 MB and uploads 1.2 MB for it, and with the proofs a chat request moves about 28 MB in all (an image request about 26 MB). Cold, the first token takes about 14-16 s on a good home connection, about a minute on a slow one. While you send confidential requests, routstrd pre-warms sessions in the background (each costs the same download, used or not); a pre-warmed request gets its first token in about 2-3 s, mostly provider latency. The answer then streams at normal speed. Image requests are never pre-warmed, so each one pays the cold cost on top of the provider's generation time.
- **Money:** from your usual balance with that node, at the rates it signed. The node holds back the maximum (a full-context prompt, your token limit and a small margin) and charges the usage the provider reported; an image is charged the flat per-image price in the signed offer. Its signed offer may add a session fee per request and an abandon fee for a session that never reached the provider (such as an unused pre-warmed one); otherwise such a session, or a provider refusal (4xx, 503), costs nothing. If the provider got your request but the session ends without its usage report (another 5xx, a cut stream), the node charges the whole reservation, and you cannot prove who cut it.
- **Providers:** chat completions and image generation, on providers that behave like Venice (others need checking). If this mode is unavailable, a confidential request is refused, never sent openly.
- **Not covered:** the provider still sees your prompt; the node can refuse service or keep money after a failed session (reputation is your protection). The two-party computation is new and has no published proof (it needs external review); a cheating node could learn at most about 2.6 bits per session about single-use values.